

DSL Project MorseML (STM32F429)

Overview

Basically, this project listens to you if you communicate using Morse code and subsequently deciphers the input. The only hardware required is a push button (Blue Button on STM32 board).

This project decodes Morse code using:

- STM32F429 (bare-metal programming, no HAL)
- TensorFlow Lite Micro (embedded CNN)
- USART (PuTTY output)

The system captures button press timings, converts them into a feature vector, and uses a trained CNN model to predict the corresponding alphabet.

What is Morse Code?

Morse code is a method of encoding text using:

- Short signals $\rightarrow$ DOT ($\cdot$)
- Long signals $\rightarrow$ DASH ($-$)

These are represented using:

- Button press duration $\rightarrow$ DOT or DASH
- Gap between presses $\rightarrow$ separation between symbols

Morse Representation of Supported Letters

The following letters are supported in this project:

- A: dot dash (short, long)
- B: dash dot dot dot (long, short, short, short)
- C: dash dot dash dot (long, short, long, short)
- D: dash dot dot (long, short, short)
- E: dot (short)

How STM32 Interprets Morse

The STM32 does not directly detect dots and dashes.

Instead, it measures:

- Press duration $\rightarrow$ press[i]
- Gap duration $\rightarrow$ gap[i]

Example:

```
A (· {}):  
Press1 100ms  
Gap1 100ms  
Press2 300ms
```

Converted into feature vector:

```
[100, 100, 300, 0, 0]
```

How ML Model Uses Morse

The ML model does not see symbols ($\cdot$ or $-$).

It only sees:

```
[press1, gap1, press2, gap2, press3]
```

This makes the system robust to:

- Different typing speeds
- Slight human errors

How to Add Morse to the Model

Step 1: Define Pattern

Each letter must be converted into a numerical pattern:

```
A  $\rightarrow$  [100,100,300,0,0]  
B  $\rightarrow$  [300,100,100,100,100]  
C  $\rightarrow$  [300,100,300,100,0]  
D  $\rightarrow$  [300,100,100,0,0]  
E  $\rightarrow$  [100,0,0,0,0]
```

Step 2: Generate Dataset

Run:

```
python dataset_generator.py
```

This:

- Adds noise ($\pm 10\text{ms}$)
- Creates multiple samples per letter
- Improves generalization

Step 3: Train Model

```
python train_cnn.py
```

This trains a CNN classifier that maps:

timing $\rightarrow$ letter

Step 4: Convert to Embedded Model

```
python convert_to_tflite.py  
python convert_to_c_array.py
```

Note: Instead of using `xxd`, a Python script is used to convert the model into a C header file compatible with STM32.

How STM32 Uses This

1. User presses button in Morse
2. STM32 records timings
3. Converts to:

```
[press1, press2, press3, ...]
```

4. Normalizes:

```
input_data[i] = press[i] / 1000.0;
```

5. Feeds into CNN
6. Outputs:

Predicted: A

Important Design Decisions

Why not detect dots and dashes manually?

- Humans are inconsistent
- Timing varies
- Hard thresholds fail

Why ML?

- Learns patterns automatically
- More robust to variation
- Easier to extend

Improving Accuracy

1. Add More Data

```
for _ in range(1000):
```

2. Add More Noise

```
noise = np.random.normal(0, 20, 5)
```

3. Normalize Better

```
X = X / np.max(X, axis=1, keepdims=True)
```

4. Add Gap Features (Advanced)

```
[press1, gap1, press2, gap2, press3]
```

Hardware Setup

- Board: STM32F429ZITx
- Button: PC13 (User Button)
- USART1:
 - PA9 → TX → PuTTY
 - PA10 → RX (unused)

PuTTY Configuration

- Baud Rate: 115200
- Data Bits: 8
- Stop Bits: 1

Example Output

```
Morse CNN Ready  
Predicted: A  
Predicted: B  
Predicted: E
```

Future Improvements

- Add full alphabet A–Z
- Add word decoding
- Add LCD output
- Real-time signal graph
- BLE or WiFi input

Summary

This project demonstrates:

- Embedded systems (bare-metal STM32)
- Signal processing (timing extraction)
- Machine Learning (CNN)
- Edge AI deployment (TFLite Micro)

A complete embedded and AI system.